Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build index receipt events and reconcile records for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Event ID, digest and timestamp are decoded from actual EVM receipts and reconciled with the business evidence list. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Review prompt

Review every external function for authorization, state transitions, integer units and invariants. Point to exact lines. Produce a requirement-to-assertion matrix and add a failing test before repairing any defect.

Scenario-specific generation constraints

Read synthetic record_id, hash as bytes32 and record it with canonical digest. Wait for receipt then decode Evidence event. Require one log, matching digest and recorded(id)=true. Reject duplicate record. Inspect indexed-events.json and reconcile its block and digest to mock input.

Expected assertions: One decoded event; exact input digest; on-chain recorded true; duplicate false attempt.

Extension prompt

Add a second mock record, index both events in block order and flag a tampered local digest. Return the exact changed source lines and positive/negative tests.